

✓ Дерево принятия решений

На предыдущих занятиях мы познакомились с некоторыми методами машинного обучения - линейной и логистической регрессиями. У этих, и у многих других методов, есть недостаток - результаты метода слабо понятны обычному человеку, или как говорят *слабо интерпретируемы*.

Действительно как объяснить человеку что

$0.5 * (\text{рост}) + 0.9 * (\text{вес}) - 0.013 * (\text{есть хвост}) > 5$ это слон, а

$0.5 * (\text{рост}) + 0.9 * (\text{вес}) - 0.013 * (\text{есть хвост}) < 5$ это собака?

Как вообще можно складывать величины разной размерности рост в метрах и вес в килограммах, на уроках физики нам говорили что так нельзя, а здесь в линейной регрессии именно так и происходит. Хотя эти методы могут давать хорошие результаты, но объяснимость их очень маленькая.

Человек думает и принимает решения по другому. ЕСЛИ сегодня выходной ТОГДА я не пойду в школу. ЕСЛИ мне поставят оценку больше 3 ТОГДА я буду хорошим учеником ИНАЧЕ плохим.

Подобным образом на основе правил вида "ЕСЛИ ... ТОГДА ... ИНАЧЕ" (IF...THEN...ELSE...) рассуждает человек и это ему понятно. Вот бы и компьютер заставить думать также. Хм, но именно так мы пишем программы для компьютера, знакомая конструкция любого языка программирования, не правда ли?

Одного правила для принятия решения может быть не достаточно - тогда можем применить несколько правил, например:

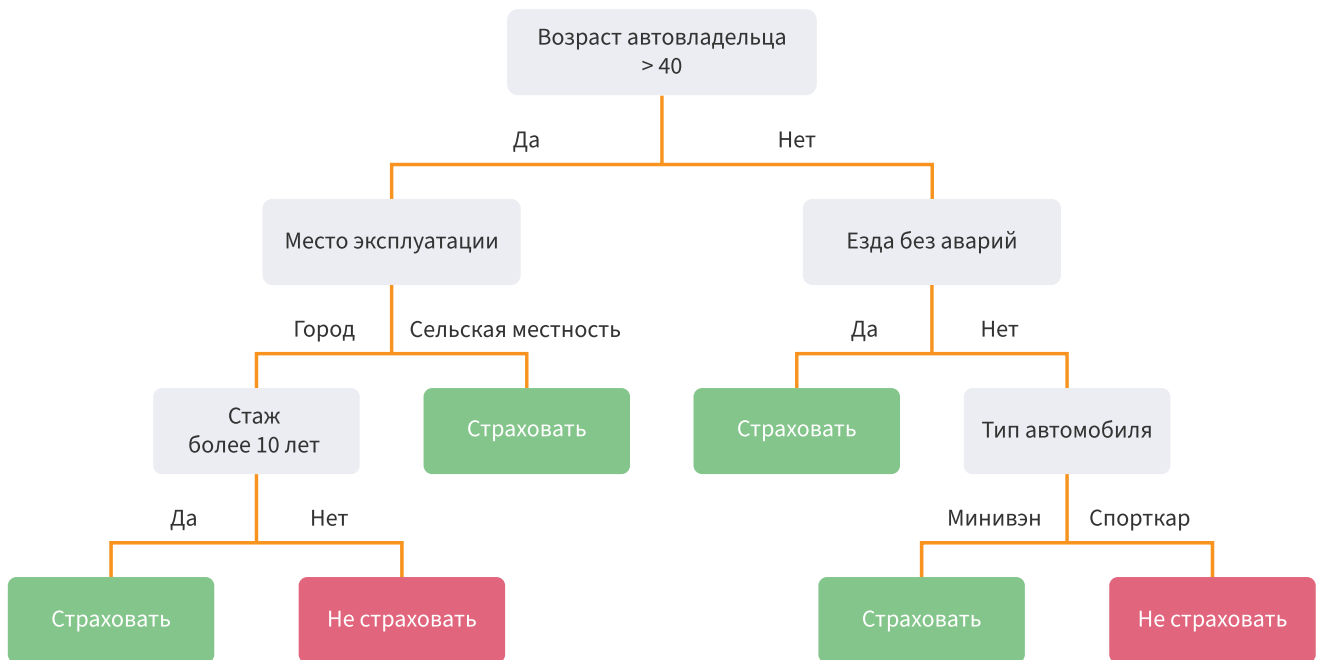
```
ЕСЛИ оценка больше 4
ТОГДА отдыхать
ИНАЧЕ ЕСЛИ оценка больше 3
----- ТОГДА немножко подучить
----- ИНАЧЕ ЕСЛИ оценка больше 2
----- ТОГДА учиться очень усердно
----- ИНАЧЕ придется отчисляться
```

Раз мы можем написать такие правила для нашего решения, тогда что мешает это сделать компьютеру?

Для любой задачи, например классификации чего-нибудь, можем построить такие правила в виде некоторой иерархии. Пример на рисунке ниже.

Используя некоторую информацию об автовладельцах - атрибуты или признаки - страховая компания принимает решение страховать или нет такого автовладельца.





Изображения иерархии подобных правил напоминают изображение дерева (перевернутого вверх корнем), поэтому так и называются - **дерево принятия решений**. Места разветвлений - узлы, конечные узлы, которые уже не разветвляются - листья. Самый первый (верхний) узел - корень дерева. Это просто названия и никакой связи с биологическими деревьями нет. В принципе деревья могут быть разной структуры, но мы будем говорить только о деревьях с ответами правил типа ДА\НЕТ.

✓ Обучение (создание) дерева

Когда дерево уже кем-то создано, мы можем им пользоваться без проблем, проверяя правила и таким образом выискивая подходящее решение.

Но дерево нужно создать, обучить. Для этого придумано множество методов, мы рассмотрим только один метод **"CART"** (Classification and Regression Trees) реализованный в библиотеке `sklearn`.

Пусть, как и обычно в задачах классификации с учителем, имеется L обучающих векторов примеров входов x^l с n компонентами (атрибутами) каждый и столько же меток y описывающих класс, к которому относится каждый из примеров. Будем рассматривать только правила с ответами ДА/НЕТ и условиями вида $x_i^l \leq t_i$, t - некоторые пороги для атрибутов.

Для построения каждого узла дерева возьмем один из атрибутов (компонент входа) и сравним его с порогом. Разделим все обучающие примеры Q этого узла на две части, выборки, в одну Q_l (левую) войдут примеры у которых условие $x_i^l \leq t_i$ выполняется, а в другую Q_r (правую) примеры у которых условие не выполняется. Посчитаем для этих двух выборок некоторую величину $H()$, называемую **неопределенность** (impurity, еще называют ее "загрязненность", "неточность", "критерий") и общую неопределенность узла

$$G = (\text{число примеров в } Q_l) / (\text{общее число примеров в } Q) * H(Q_l) + (\text{число примеров в } Q_r) / (\text{общее число примеров в } Q) * H(Q_r).$$

И сделаем это для всех атрибутов и всех возможных значений порога этих атрибутов. Выберем такой атрибут и порог, для которых G минимально. По ним и будем окончательно строить узел.

Начиная с корня, в котором присутствуют все обучающие примеры x^l , будем создавать новые и новые узлы, постепенно разбивая набор этих примеров на меньшие части по выбранным атрибутам и порогам

пока не останется в каждом узле по одному примеру, или не выполняются другие критерии останова обучения.

Так и получится дерево решений.

Реализовано несколько разных критериев $H()$. Такой критерий должен давать маленькое число, если мы смогли за одно разделение отделить классы полностью (идеальное разделение) и большое, если число примеров разного класса в разделенных множествах примерно одинаково (бесполезное разделение), например:

А) Для задач классификации на K классов:

1) *Неопределенность Джини*: для каждого класса считаем долю (p) примеров этого класса во всех примерах (их N_m штук) узла m (метки y представлены целыми числами от 0 до $K-1$) и считаем $H()$ как

$$p_k = \frac{1}{N_m} * \sum_{x_i \in Q_m} I(y_i = k), I = 1 \text{ если совпадает, } 0 \text{ если не совпадает}$$

$$H = \sum_k p_k * (1 - p_k)$$

2) *Энтропия*:

$$H = - \sum_k p_k * \log(p_k)$$

3) *Ошибка классификации*:

$$H = 1 - \max(p_k)$$

Б) Для задач регрессии обычно используют среднеквадратичную ошибку: сначала ищут среднее значение меток y в примерах узла m

$$y_{mean} = \frac{1}{N_m} \sum_{i \in N_m} y_i$$

а потом ищут среднеквадратичную ошибку

$$H = \frac{1}{N_m} \sum_{i \in N_m} (y_i - y_{mean})^2$$

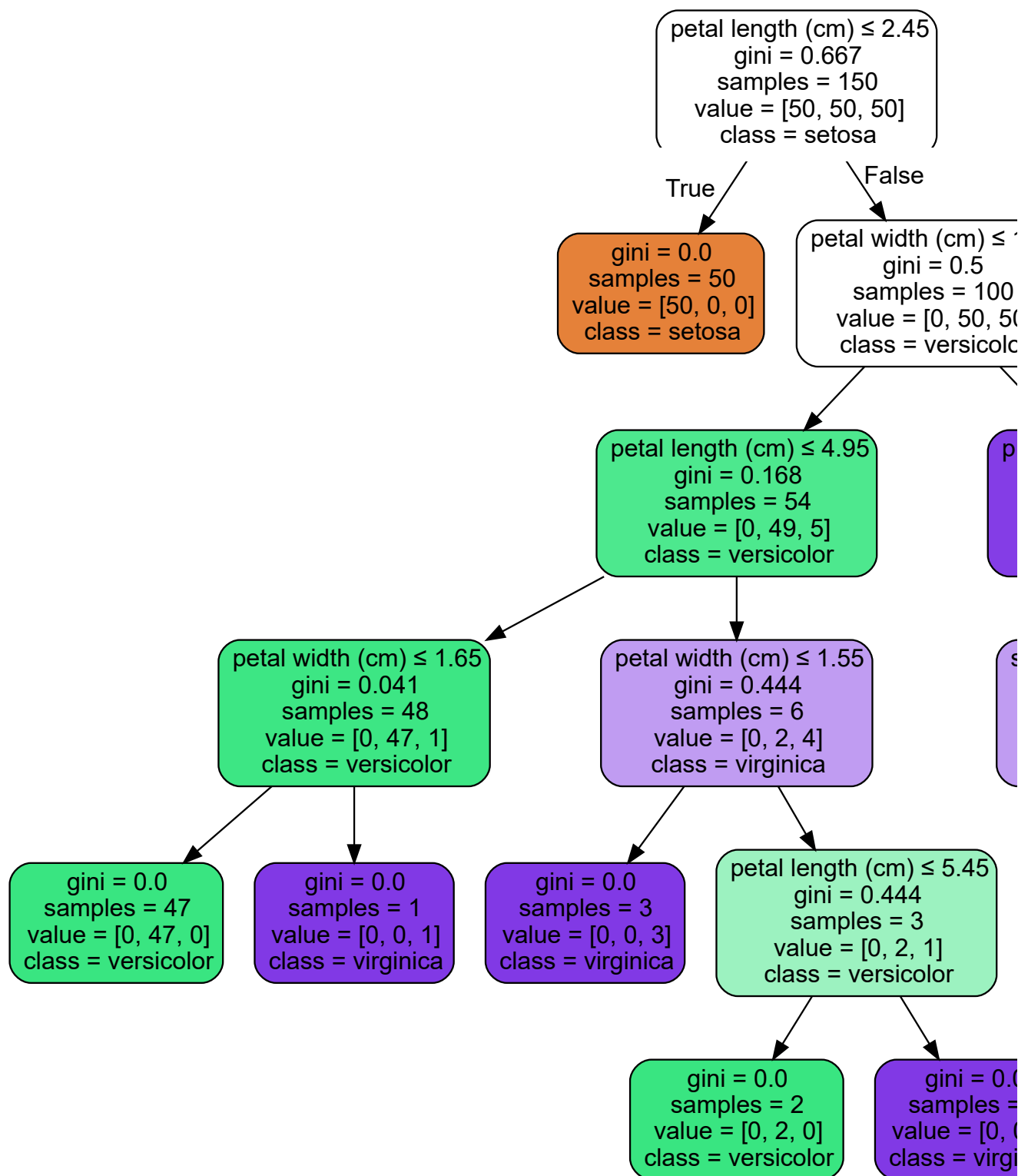
Прекрасную анимацию создания дерева можно найти [здесь](#).

Давайте реализуем классификатор на основе дерева для примеров [ирисов Фишера](#). В библиотеке `sklearn` есть модуль `tree` с различными функциями для деревьев, используем оттуда [DecisionTreeClassifier\(\)](#) для создания классификатора. Синтаксис его аналогичен другим классификаторам: команда `fit()` для обучения, `predict()` для расчета выходов. Для отображения дерева используем команду `plot_tree()` (она вернет и текстовое описание дерева), которая покажет для каждого узла выбранный атрибут (как индекс входного массива), порог, число примеров в узле общее (`samples`) и по классам (`values`), значение критерия (здесь `gini`).

```
from sklearn.datasets import load_iris # примеры данных
from sklearn import tree # модуль для деревьев
iris = load_iris() # загружаем данные
X=iris.data # примеры входов
y=iris.target # метки (примеры выходов)
clf = tree.DecisionTreeClassifier() # создаем классификатор на основе дерева
clf = clf.fit(X, y) # обучаем его, т.е. создаем само дерево
tree.plot_tree(clf) # отображаем
```



```
graph = graphviz.Source(dot_data) ## загружаем дерево из переменной или файла в представление graphviz
graph ## отображаем на экране
```



Функция `export_text()` из того же модуля `sklearn.tree` позволяет отобразить дерево в форматированном текстовом виде правил ЕСЛИ...ТОГДА...ИНАЧЕ

```
from sklearn.tree import export_text # подключаем функцию
r = export_text(clf, feature_names=iris['feature_names']) # переводим дерево в текстовую строку
print(r) # печатаем
```



```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
```

```

| | | |--- petal width (cm) <= 1.75
| | | |--- petal length (cm) <= 4.95
| | | |--- petal width (cm) <= 1.65
| | | |--- class: 1
| | | |--- petal width (cm) > 1.65
| | | |--- class: 2
| | | |--- petal length (cm) > 4.95
| | | |--- petal width (cm) <= 1.55
| | | |--- class: 2
| | | |--- petal width (cm) > 1.55
| | | |--- petal length (cm) <= 5.45
| | | |--- class: 1
| | | |--- petal length (cm) > 5.45
| | | |--- class: 2
| | |--- petal width (cm) > 1.75
| | |--- petal length (cm) <= 4.85
| | |--- sepal width (cm) <= 3.10
| | |--- class: 2
| | |--- sepal width (cm) > 3.10
| | |--- class: 1
| | |--- petal length (cm) > 4.85
| | |--- class: 2

```

✓ Разделяющая поверхность

Раз деревом можно решать задачу классификации, то оно реализует какую-то разделяющую поверхность. Зная устройство дерева подумайте немного, как выглядит эта поверхность (на двумерном примере проще всего).

Подумали? Теперь выполните код ниже, чтобы убедиться, что разделяющая поверхность для дерева в двумерном случае это набор прямоугольников, возможно вырожденных в линию или точку.

Чтобы можно было отобразить на плоскости, будем строить несколько деревьев для пар атрибутов и рисовать разделяющие поверхности.

```

import numpy as np #
import matplotlib.pyplot as plt #

from sklearn.datasets import load_iris #
from sklearn.tree import DecisionTreeClassifier, plot_tree #

# Parameters
n_classes = 3 # число классов
plot_colors = "ryb" # цвета
plot_step = 0.02 # шаг для симуляции на плоскости

# Load data
iris = load_iris() # загружаем данные

# в цикле по числу пар атрибутов
for pairidx, pair in enumerate([[0, 1], [0, 2], [0, 3],
                                [1, 2], [1, 3], [2, 3]]):

    X = iris.data[:, pair] # Выбираем заданные пары атрибутов
    y = iris.target # метки

    # Обучение
    clf = DecisionTreeClassifier(max_depth=2).fit(X, y) # обучаем дерево на текущей паре атрибутов

    # Строим графики
    plt.subplot(2, 3, pairidx + 1) # подграфик для текущей пары

```

```

x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1 # немного измененные минимальное и максимальные значения
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1 # немного измененные минимальное и максимальные значения
# считаем прямоугольную сетку возможных значений этих атрибутов
xx, yy = np.meshgrid(np.arange(x_min, x_max, plot_step), #
                     np.arange(y_min, y_max, plot_step)) #
plt.tight_layout(h_pad=0.5, w_pad=0.5, pad=2.5) # более компактный график

# считаем выход классификатора для всех примеров сетки
# не забыв что массивы данных нужно привести к требуемому размеру.
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape) # и преобразуем обратно к исходному размеру
cs = plt.contourf(xx, yy, Z, cmap=plt.cm.RdYlBu) # рисуем контурную карту

plt.xlabel(iris.feature_names[pair[0]]) # добавляем подписи осей
plt.ylabel(iris.feature_names[pair[1]]) #

# Отображаем обучающие примеры своим цветом
for i, color in zip(range(n_classes), plot_colors): # в цикле по количеству классов
    idx = np.where(y == i) # отбираем все точки текущего класса
    # отображаем их своим цветом
    plt.scatter(X[idx, 0], X[idx, 1], c=color, label=iris.target_names[i], #
               cmap=plt.cm.RdYlBu, edgecolor='black', s=15) #

plt.suptitle("Decision surface of a decision tree using paired features") # подписываем график
plt.legend(loc='lower right', borderpad=0, handletextpad=0) # легенда
plt.axis("tight"); # отображение осей

```

```
# построим и отобразим дерево для всех 3 классов.
plt.figure() #
clf = DecisionTreeClassifier().fit(iris.data, iris.target) #
plot_tree(clf, filled=True) #
plt.show() #
```



Еще более красивая и наглядная визуализация деревьев возможна с помощью библиотеки [dtreeviz](#). Но мы оставим ее изучение на самостоятельную работу.



✓ Замечание о важности признаков

Дерево решений дает возможность посчитать "важность" признаков, их вклад в решение, см. поле `feature_importances_`.

Важность можно считать и для других моделей, например логистической регрессии, где [модуль веса можно интерпретировать как важность](#) (при условии, что признаки нормированы).

```
clf.feature_importances_
```



```
array([0.01333333, 0.01333333, 0.05072262, 0.92261071])
```

Задание и обсуждение

Попробуйте изменять параметры дерева и смотрите как это влияет на качество решения задачи. Попробуйте на своих данных.